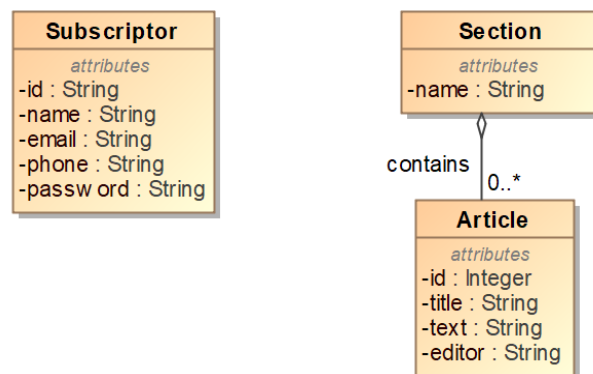


Análisis y Diseño con Patrones

PEC 2: Patrones de Diseño y de Asignación de Responsabilidades

Trabajamos para un periódico digital proporcionando soporte técnico. El periódico distribuye sus artículos entre un conjunto de secciones. Un artículo pertenece a una única sección. Se accede al contenido de todas sus secciones a través de suscripción. Para ser suscriptor solicitamos un identificador, nombre, email, teléfono móvil y contraseña.

Comenzamos el diseño partiendo del siguiente diagrama de análisis estático de clases:



Pregunta 1 (20%)

Queremos añadir funcionalidades permitiendo que los suscriptores puedan gestionar sus comentarios en los artículos. Un comentario contiene un id de valor entero (que establece su posición u orden de creación, dentro del conjunto de comentarios emitidos por los suscriptores sobre un artículo en concreto), el id del suscriptor que lo realiza, la fecha y el contenido del comentario.

Estas nuevas funcionalidades, que dan soporte a comentarios, deben ofrecer la posibilidad de añadir comentarios a un artículo, recuperarlos a partir de su posición y borrarlos de forma individual a través de su posición dentro del conjunto, y también de forma grupal a través del id del suscriptor (borrará todos los comentarios sobre el artículo realizados por el suscriptor).

Se pide:

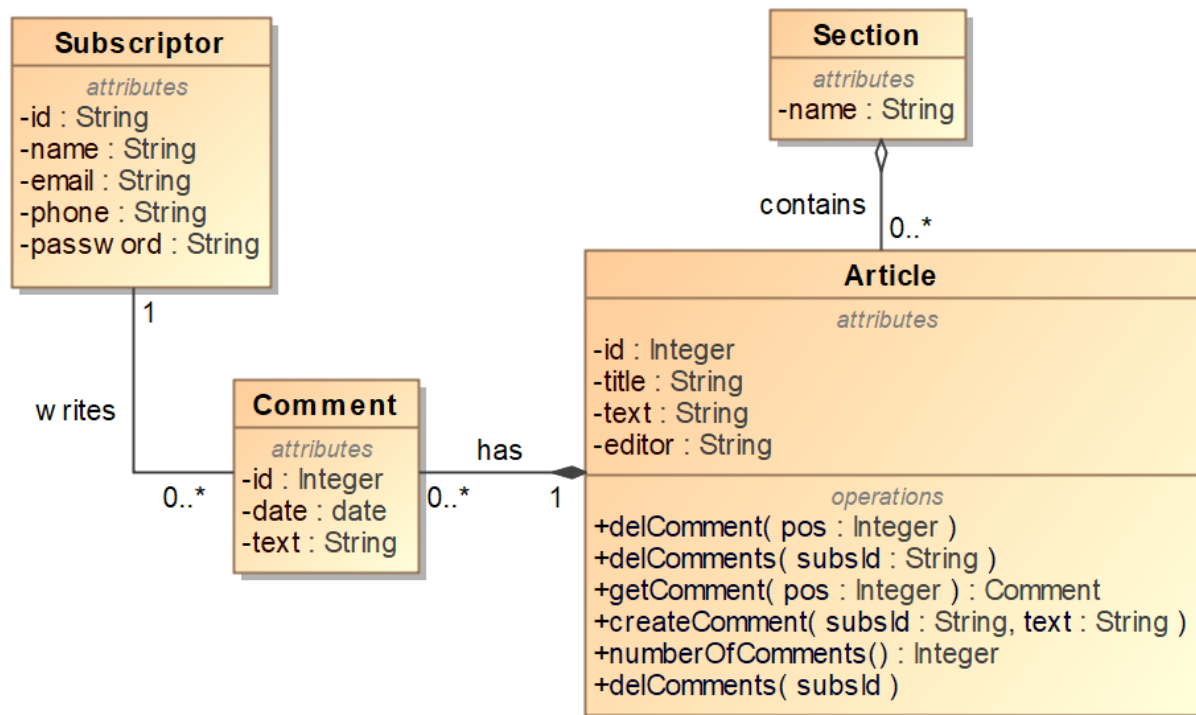
- Justificar qué patrones de responsabilidad y diseño se deben aplicar para incorporar estas nuevas funcionalidades.
- Presentar el diagrama de clases incluyendo la solución de soporte a comentarios.

Solución:

- **Creador:** ya que corresponde al artículo crear y contener los comentarios realizados por los suscriptores.

Experto: porque al artículo, al contener la información necesaria para las nuevas funcionalidades, le corresponde la responsabilidad de gestionar los comentarios que contiene.

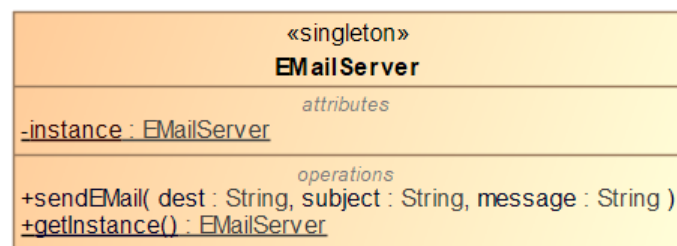
- **Diagrama estático de clases:**



Pregunta 2 (30%)

Añadimos a nuestros suscriptores un servicio de notificaciones. Los suscriptores podrán escoger sus secciones favoritas. Cuando un nuevo artículo sea publicado en alguna de ellas, se les enviará un email con el título del mismo. Queremos proporcionar una solución que evite el acoplamiento entre los suscriptores y los artículos.

Disponemos de un servidor de correo EMailServer. No es necesario conocer los detalles de su configuración, ni del método sendEmail. Estos detalles quedan encapsulados en esta clase, a cuyos servicios se accede a través de la aplicación del patrón Singleton:



Se pide:

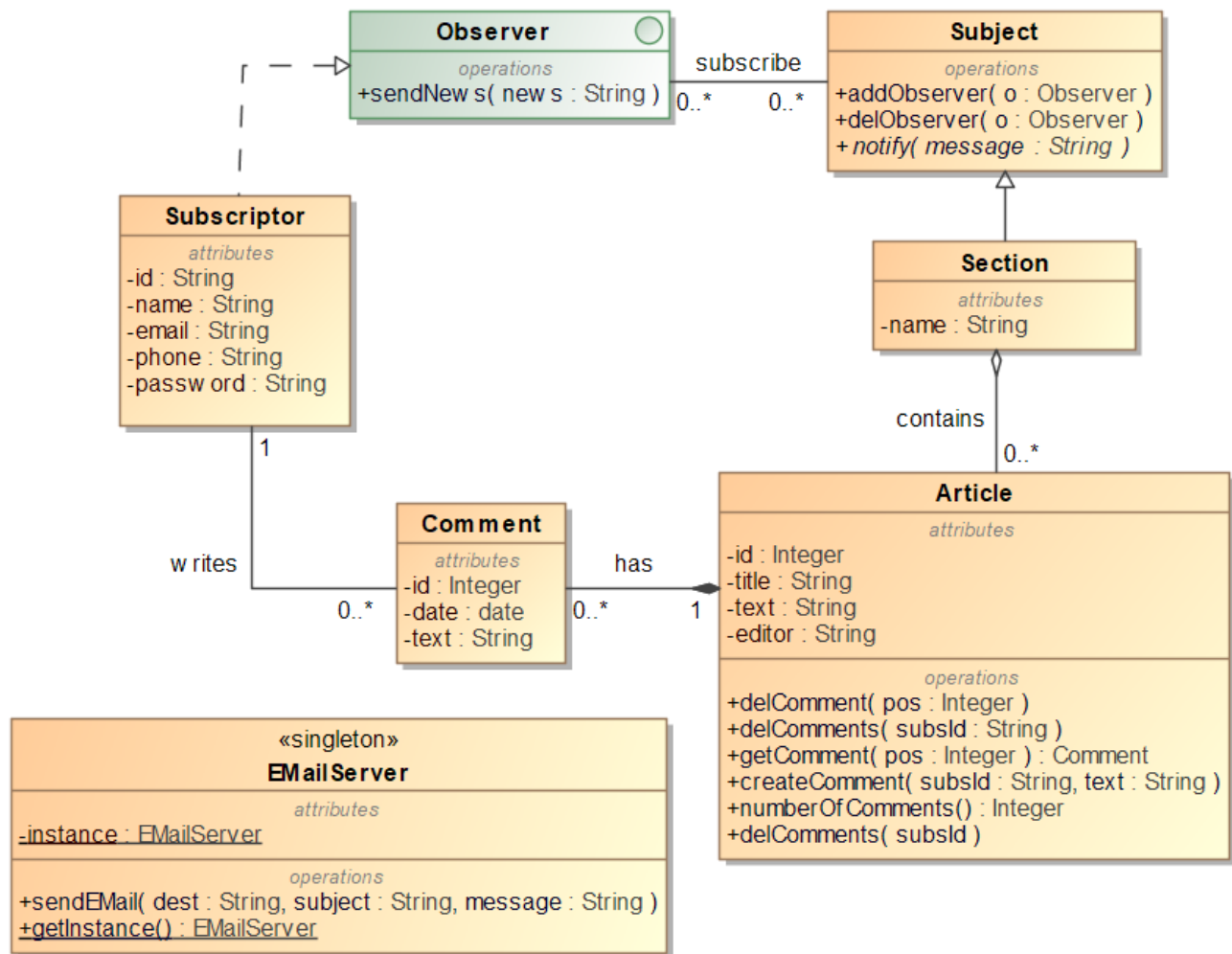
- Justificar qué patrones de responsabilidad y diseño podemos aplicar para gestionar las notificaciones.
- Modificar y presentar el diagrama estático de clases.
- Mostrar el pseudocódigo de las clases que intervienen en el servicio de notificación a los suscriptores.

Solución:

- **Observador:** Aunque el contexto teórico dice que se aplica cuando "un cambio en un objeto implica cambios en otros objetos", en los Módulos se ha mostrado que se puede aplicar a la gestión de estas notificaciones a las suscripciones. En este caso específico utilizando la primera de sus variantes, donde el objeto observado (Section) no se pasa a sí mismo como parámetro al servicio que realiza la notificación. Section notifica a su lista de suscriptores el título de cada nuevo artículo que recibe.

Además, se puede argumentar la aplicación del patrón **Experto** debido a que la notificación no la realizarán los objetos de la clase Subject ya que esta clase no tiene la información necesaria, sino los objetos de la clase derivada Section (que tienen acceso al título del artículo).

- **Diagrama estático de clases:**



· Pseudocódigo de las clases involucradas en el servicio de notificación:

```

public class Subject {

    protected List<Observer> observers;

    public void addObserver (Observer o) {

        observers.add(o);

    }

    public void delObserver (Observer o) {

        observers.remove(o);

    }

}

```

```
}

public abstract void notify (String message) {

}

}

public class Section extends Subject {

    protected List<Article> articles;

    public void addArticle (Article article) {

        articles.add(article);

        String title = article.getTitle();

        notify(title);

    }

    public void notify (String message) {

        foreach (observer in observers) {

            observer.sendNews(message);

        }

    }

}

public class Subscriptor implements Observer {

    public void sendNews (String message) {

        EmailServer emailServer = EmailServer.getInstance();

        emailServer.sendEmail(this.getEmail(),

            "news from your favorite Section: ", message);

    }

}
```

Pregunta 3 (20%)

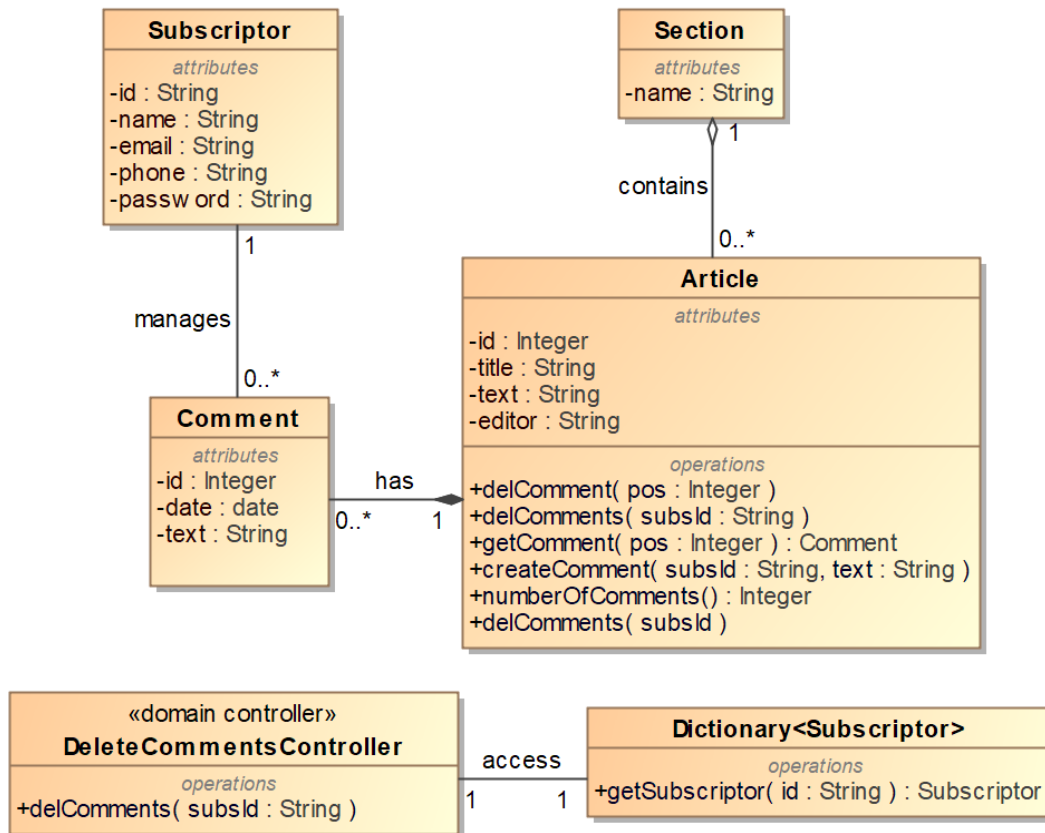
Se puede dar el caso de que se detecte la presencia de comentarios inadecuados emitidos por un suscriptor. A partir de la solución adoptada en la Pregunta 1, debemos desarrollar un servicio de la capa de dominio que permita a un administrador eliminar todos los comentarios asociados a un suscriptor.

Sin modificar las clases existentes ni sus relaciones, se pide:

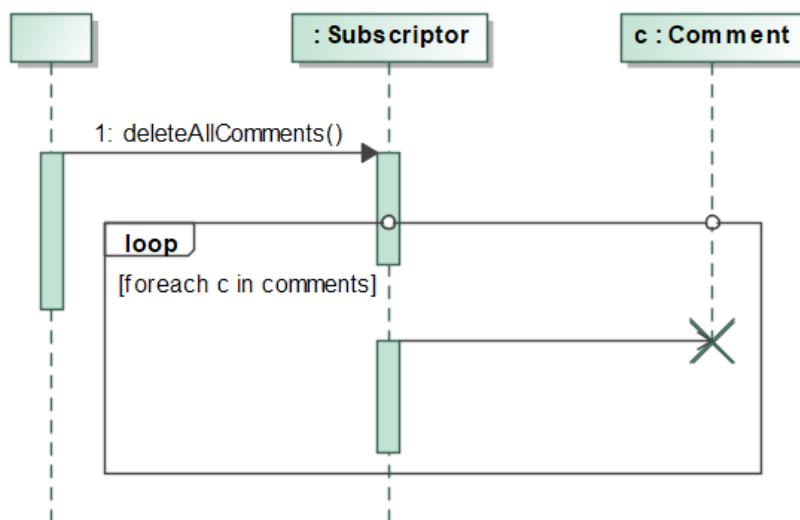
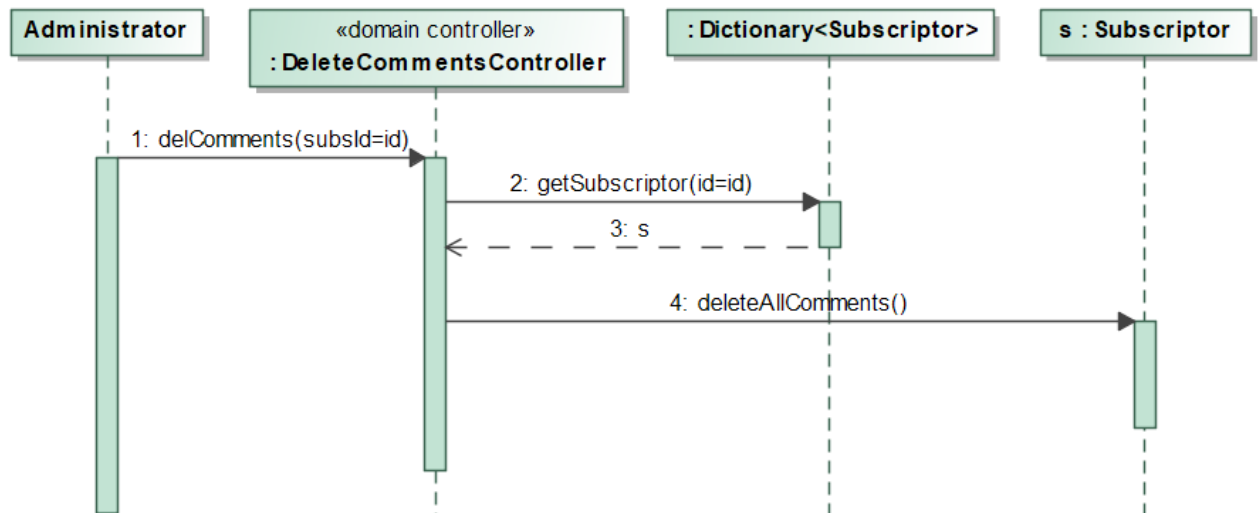
- Indicar justificadamente qué patrones de responsabilidad y diseño debemos aplicar para implementar este servicio de la capa de dominio.
- Mostrar cómo quedaría integrado este servicio dentro del diagrama de clases.
- Mostrar el diagrama de secuencia de este servicio.

Solución:

- **Patrón controlador**, necesitamos una clase que asuma la responsabilidad de gestionar el servicio que proporcionará la capa de dominio. Este controlador recibirá de la vista el id del suscriptor. A través de un diccionario de suscriptores obtendremos el objeto Suscriptor asociado al id, y se procederá a eliminar todos los comentarios de dicho suscriptor.
- **Mostrar cómo quedaría integrado este servicio dentro del diagrama de clases.**



- Mostrar el diagrama de secuencia de este servicio.



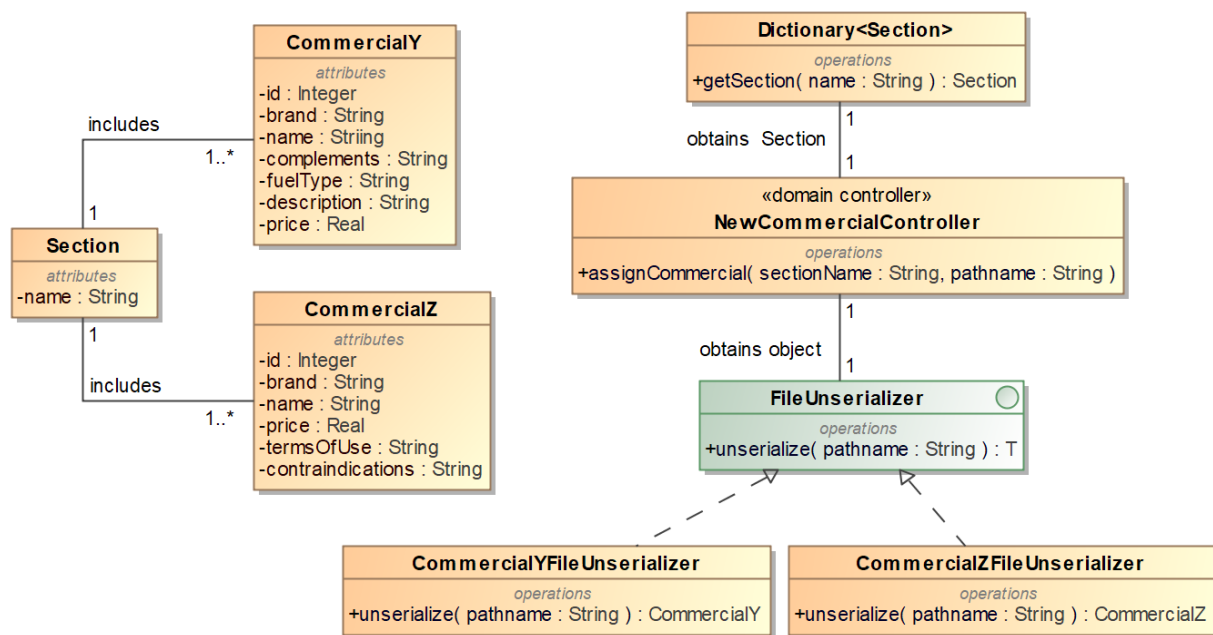
Pregunta 4 (30%)

Nos piden incluir anuncios comerciales en cada una de nuestras secciones. De estos anuncios nos interesa obtener: el identificador del anuncio, la marca, el modelo, el precio y una descripción de lo que se anuncia y sus características.

Contexto: Los anunciantes no tienen acceso a nuestra aplicación. Nos envían un fichero XML con los datos del anuncio y el nombre de la sección donde debe incluirse. Un controlador de dominio recibe estos datos, obtiene un objeto que representa al anuncio y lo añade a la Sección especificada. Para generar objetos a partir de un fichero XML utilizamos el framework JAXB.

Comenzamos nuestro diseño y nos encontramos con que cada empresa anunciante nos va a proporcionar información diferente para sus anuncios. Abandonamos nuestra idea original para adaptarnos a la información proporcionada por cada anunciante.

Por ejemplo, en el siguiente diagrama de clases se muestran 2 anuncios diferentes y como se ha gestionado la transformación de ficheros XML a objeto, a través de clases especializadas en cada tipo de anuncio y una interfaz que retorna un tipo genérico:



Pseudocódigo del método assignCommercial del controlador de dominio NewCommercialController:

```

public class NewCommercialController {

    private Dictionary<Section> sections;

    private FileUnserializer unserializer;

    // unserializer es inyectado vía constructor en función
  
```

```
// del anunciante propietario del fichero

public void assignCommercial(String sectionName, String pathname) {

    Section s = sections.getSection (sectionName);

    switch (typeof(unserializer)) {

        case CommercialYFileUnserializer:

            CommercialY comY = unserializer.unserialize(pathname);

            s.addCommercialY(comY);

        case CommercialZFileUnserializer:

            CommercialZ comZ = unserializer.unserialize(pathname);

            s.addCommercialZ(comZ);

    }

}
```

Nos damos cuenta que el diseño propuesto tiene diferentes problemas:

- 1) El acoplamiento que se genera entre la clase Section y los diferentes anuncios que nos proporcionan las empresas.
- 2) El controlador está acoplado a los tipos de objeto que cubre la interfaz, a los anuncios y a las listas de anuncios que contiene Section. Tampoco cumpliremos open-close cuando se añadan nuevos anunciantes.

Conclusión: No se ha sabido gestionar desde el principio el exceso de campos de información que nos proporcionan las empresas, que no podemos evitar, y arrastramos un alto acoplamiento por todo el diseño.

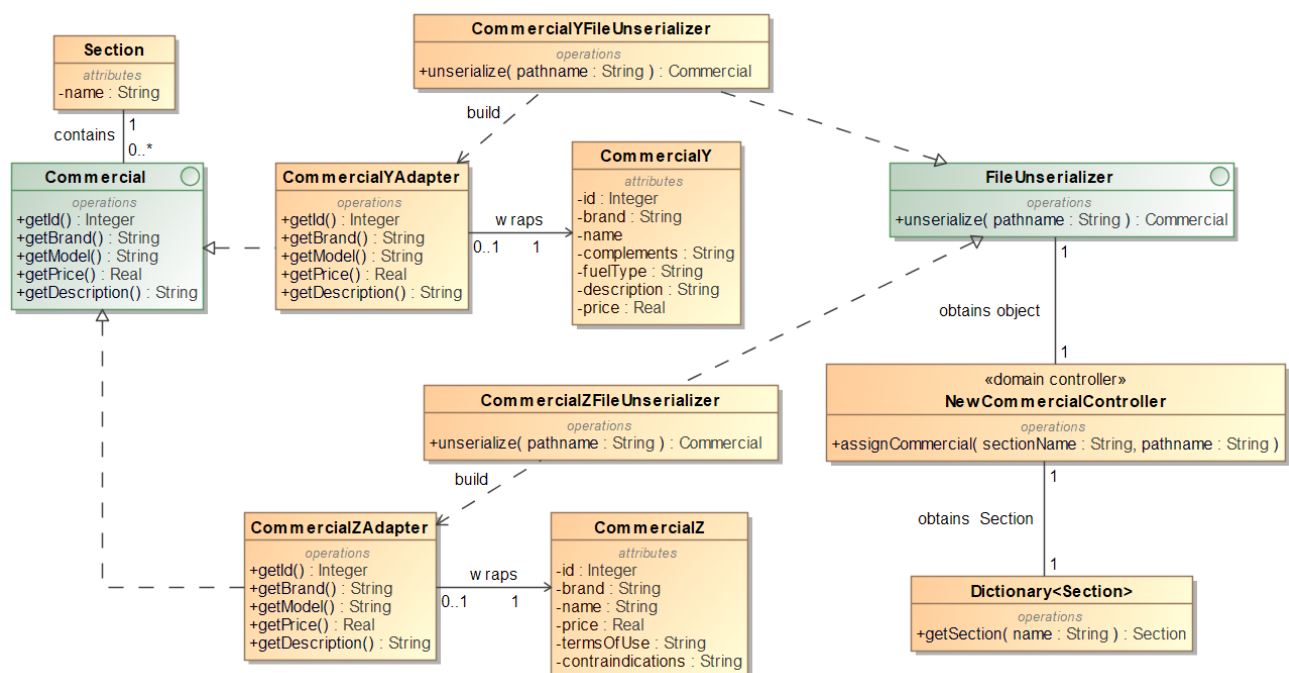
Queremos volver a la idea original, unificando la gestión de los anuncios bajo la interfaz común descrita al comienzo del enunciado. Desacoplando así tanto a Section como al controlador de dominio de los diferentes tipos de anuncios.

Se pide:

- Justificar qué patrones de diseño debemos aplicar para resolver los problemas mencionados.
- Mostrar su aplicación en un nuevo diagrama estático de clases, centrado en las clases mostradas en el enunciado del ejercicio.
- Mostrar el pseudocódigo del método assignCommercial tras solucionar los problemas.

Solución:

- **Patrón de diseño Adapter:** “Permite utilizar una clase utilizando un conjunto de operaciones diferente al que ofrece originalmente”. Lo aplicaremos a todas aquellas clases que representan un tipo de anuncio cuya interfaz pública difiera de la que queremos utilizar para simplificar la gestión de los anuncios.
- **Diagrama estático de clases parcial, centrado en las clases de este ejercicio:**



- **Mostrar el pseudocódigo del método assignCommercial.**

```
public void assignCommercial (String sectionName, String pathname) {  
    Section s = sections.getSection (sectionName);  
    Commercial commercial = unserializer.unserialize(pathname);  
    s.addCommercial(commercial);  
}
```